



AdaLoRA: Adaptive Rank Allocation for Parameter- Efficient Fine-Tuning

PRESENTED BY BUYI GENG

04/29/2025

Hook

- Same LoRA budget – up to +4.1 pp accuracy with –40 % trainable params
- Finetune GPT-2/BLOOM/T5 on one consumer GPU and still beat full LoRA
- Big idea: spend params where gradients say they matter

Pain Point in PEFT

- Uniform rank r for every layer
 - Ignores layer importance variance
 - Wastes budget on low-impact layers
- Need adaptive budget per layer

LoRA refresher

1. $\Delta W = A \cdot B$ (rank r); train A & B only
2. Pros: tiny params; backbone frozen
3. Cons: manual r , same for all layers

Idea #1 – Importance scoring

- Score $S = \|g \odot W\|_2$
- g : gradient, W : weight
- High $S \rightarrow$ layer sensitive to change
- Warm-up phase collects S

Idea #2 – Dynamic rank scaling

- Adapter = $P \Sigma Q^T$ (trainable SVD)
- Grow high-S layers, prune low-S layers
- Three knobs: budget K , scores S , initial r_0

Idea #3 – Progressive budget shrink

- Stage 1 Explore: big r_0 , collect scores
- Stage 2 Compress: cubic decay to K_{final}
- Stage 3 Freeze: fine-tune with fixed ranks

Algorithm Flow

Algorithm 1 AdaLoRA

```
1: Input: Dataset  $\mathcal{D}$ ; total iterations  $T$ ; budget schedule  $\{b^{(t)}\}_{t=0}^T$ ; hyperparameters  $\eta, \gamma, \beta_1, \beta_2$ .
2: for  $t = 1, \dots, T$  do
3:   Sample a mini-batch from  $\mathcal{D}$  and compute the gradient  $\nabla \mathcal{L}(\mathcal{P}, \mathcal{E}, \mathcal{Q})$ ;
4:   Compute the sensitivity  $I^{(t)}$  in (8) for every parameter in  $\{\mathcal{P}, \mathcal{E}, \mathcal{Q}\}$ ;
5:   Update  $\bar{I}^{(t)}$  as (9) and  $\bar{U}^{(t)}$  as (10) for every parameter in  $\{\mathcal{P}, \mathcal{E}, \mathcal{Q}\}$ ;
6:   Compute  $S_{k,i}^{(t)}$  by (7), for  $k = 1, \dots, n$  and  $i = 1, \dots, r$ ;
7:   Update  $P_k^{(t+1)} = P_k^{(t)} - \eta \nabla_{P_k} \mathcal{L}(\mathcal{P}, \mathcal{E}, \mathcal{Q})$  and  $Q_k^{(t+1)} = Q_k^{(t)} - \eta \nabla_{Q_k} \mathcal{L}(\mathcal{P}, \mathcal{E}, \mathcal{Q})$ ;
8:   Update  $\Lambda_k^{(t+1)} = \mathcal{T}(\Lambda_k^{(t)} - \eta \nabla_{\Lambda_k} \mathcal{L}(\mathcal{P}, \mathcal{E}, \mathcal{Q}), S_k^{(t)})$  given the budget  $b^{(t)}$ .
9: end for
10: Output: The fine-tuned parameters  $\{\mathcal{P}^{(T)}, \mathcal{E}^{(T)}, \mathcal{Q}^{(T)}\}$ .
```

- 1 Warm-up $\rightarrow$ 2 Score & (Re)allocate $\rightarrow$ 3 Update $P/\Sigma/Q^T \rightarrow$ loop
- Stop when total ranks = K_{final}

Results: GLUE + QA

Table 1: Results with DeBERTaV3-base on GLUE development set. The best results on each dataset are shown in **bold**. We report the average correlation for STS-B. *Full FT*, *HAdapter* and *PAdapter* represent full fine-tuning, Housby adapter, and Pfeiffer adapter respectively. We report mean of 5 runs using different random seeds.

Method	# Params	MNLI m/mm	SST-2 Acc	CoLA Mcc	QQP Acc/F1	QNLI Acc	RTE Acc	MRPC Acc	STS-B Corr	All Ave.
Full FT	184M	89.90/90.12	95.63	69.19	92.40/89.80	94.03	83.75	89.46	91.60	88.09
BitFit	0.1M	89.37/89.91	94.84	66.96	88.41/84.95	92.24	78.70	87.75	91.35	86.02
HAdapter	1.22M	90.13/90.17	95.53	68.64	91.91/89.27	94.11	84.48	89.95	91.48	88.12
PAdapter	1.18M	90.33/90.39	95.61	68.77	92.04/89.40	94.29	85.20	89.46	91.54	88.24
LoRA _{r=8}	1.33M	90.65/90.69	94.95	69.82	91.99/89.38	93.87	85.20	89.95	91.60	88.34
AdaLoRA	1.27M	90.76/90.79	96.10	71.45	92.23/89.74	94.55	88.09	90.69	91.84	89.31
HAdapter	0.61M	90.12/90.23	95.30	67.87	91.65/88.95	93.76	85.56	89.22	91.30	87.93
PAdapter	0.60M	90.15/90.28	95.53	69.48	91.62/88.86	93.98	84.12	89.22	91.52	88.04
HAdapter	0.31M	90.10/90.02	95.41	67.65	91.54/88.81	93.52	83.39	89.25	91.31	87.60
PAdapter	0.30M	89.89/90.06	94.72	69.06	91.40/88.62	93.87	84.48	89.71	91.38	87.90
LoRA _{r=2}	0.33M	90.30/90.38	94.95	68.71	91.61/88.91	94.03	85.56	89.71	91.68	88.15
AdaLoRA	0.32M	90.66/90.70	95.80	70.04	91.78/89.16	94.49	87.36	90.44	91.63	88.86

- GLUE: +1.0 pp over LoRA at 0.3 M params
- SQuAD F1 88.7 with just 0.08 % params
- ≤ 1 % extra FLOPs vs full fine-tuning

Data reproduced from Tables 1–2, Zhang et al., 2023

Why it matters

- Higher accuracy, same VRAM / compute
- Removes manual rank hyper-search
- Composable with QLoRA / quant / distillation
- Inspires project: find best adapter spots

Takeaways & Conclusion

- **Pay-as-you-go:** allocate rank only where gradients provide the biggest gain
- **Same VRAM, higher accuracy:** up to **+4.1 pp** on GPT-2 / BLOOM / T5 with **-40 % trainable params**
- **No manual rank grid-search:** adaptive scheduler handles r for every layer
- **Composable:** stacks cleanly with QLoRA 4-bit, quantization, distillation, sparsity
- **Next steps:**
 1. Combine **AdaLoRA + QLoRA 4-bit**
 2. Explore **layer-selective adapters** (only key layers)
 3. Implement a **TensorFlow scratch version** for the course project

One-line summary: *AdaLoRA = LoRA $\times$ smart budget allocation — squeezing the most performance out of every parameter.*